

Laboratorium
Robotika dan Sistem Cerdas
Departemen Teknik Komputer
Institut Teknologi Sepuluh Nopember

Laporan Akhir
Praktikum Sistem Mikroprosesor dan Sistem Tertanam

Pengantar Mikroprosesor

Muhammad Pasha Ramadhan - 5024241024

2026

1 Langkah-Langkah Percobaan

1.1 Percobaan 1: Pengenalan STM32 Blackpill (Blink)

1. Mengatur konfigurasi `platformio.ini` untuk board STM32 Blackpill dengan framework Arduino.
2. Menghubungkan ST-Link ke board Blackpill melalui pin SWDIO, SWCLK, GND, dan 3.3V.
3. Menuliskan program Blink pada file `src/main.cpp`.
4. Melakukan Build dan Upload program menggunakan PlatformIO.
5. Mengamati LED onboard yang menyala dan mati secara bergantian setiap 500 ms.

1.2 Percobaan 2: Pengenalan ESP32 dengan ESP-IDF (Blink)

1. Mengubah konfigurasi `platformio.ini` menggunakan board ESP32 DevKit dan framework ESP-IDF.
2. Menghubungkan ESP32 ke komputer melalui kabel USB.
3. Membuat file `src/main.c` dan menuliskan program Blink menggunakan ESP-IDF.
4. Melakukan Build dan Upload program melalui PlatformIO.
5. Mengamati LED onboard ESP32 yang menyala dan mati secara bergantian.

1.3 Percobaan 3: Pull-up dan Pull-down Eksternal dan Internal

1. Mengatur konfigurasi PlatformIO menggunakan board ESP32 DevKit dan framework Arduino.
2. Menghubungkan dua pushbutton pada GPIO 32 dan GPIO 33 dengan resistor pull-up dan pull-down eksternal.
3. Menuliskan program untuk membaca kondisi kedua tombol, kemudian melakukan Build dan Upload.
4. Melepas resistor eksternal dan mengaktifkan konfigurasi pull-up dan pull-down internal melalui program.
5. Melakukan Build dan Upload kembali, lalu membandingkan hasil pembacaan tombol pada kedua konfigurasi.

1.4 Percobaan 4: Debouncing pada Input Tombol GPIO

1. Menghubungkan pushbutton ke GPIO 25 ESP32 dengan konfigurasi pull-up internal.
2. Menuliskan program untuk membandingkan penghitung tombol dengan dan tanpa debouncing.
3. Melakukan Build dan Upload program, kemudian membuka Serial Monitor.
4. Menekan tombol beberapa kali dan membandingkan hasil kedua penghitung.
5. Mengubah nilai debounce delay menjadi 10 ms dan 200 ms, kemudian mengamati perbedaannya.

1.5 Percobaan 5: Pembacaan ADC dengan LDR

1. Merangkai LDR dan resistor 10kΩ sebagai pembagi tegangan, kemudian menghubungkan titik tengahnya ke GPIO 34 ESP32.
2. Menuliskan program untuk membaca nilai ADC dan mengonversinya menjadi tegangan.
3. Melakukan Build dan Upload program, kemudian membuka Serial Monitor.
4. Menutup LDR dengan tangan dan menyinarinya menggunakan lampu.
5. Mengamati perubahan nilai ADC dan tegangan yang ditampilkan pada Serial Monitor.

2 Analisis Hasil Percobaan

2.1 Percobaan 1: Pengenalan STM32 Blackpill (Blink)

Program berhasil dikompilasi dan dijalankan sehingga LED onboard STM32 dapat menyala dan mati secara bergantian. Hal ini menunjukkan bahwa board dapat dikendalikan melalui program yang dibuat menggunakan PlatformIO.

2.2 Percobaan 2: Pengenalan ESP32 dengan ESP-IDF (Blink)

Pada awalnya, proses Build mengalami kegagalan karena Windows menggunakan assembler dari Strawberry Perl atau MSYS2 yang tidak sesuai dengan kebutuhan toolchain ESP32. Hal ini menyebabkan assembler tidak dapat mengenali opsi khusus untuk arsitektur Xtensa, sehingga proses konfigurasi compiler oleh CMake gagal. Setelah konflik toolchain diperbaiki dan Build dijalankan kembali, program berhasil dikompilasi. LED ESP32 kemudian dapat berkedip sesuai dengan waktu yang ditentukan.

2.3 Percobaan 3: Pull-up dan Pull-down Eksternal dan Internal

Hasil percobaan menunjukkan bahwa konfigurasi pull-up dan pull-down dapat membaca kondisi tombol dengan baik. Pull-up menghasilkan nilai LOW saat tombol ditekan, sedangkan pull-down menghasilkan nilai HIGH. Penggunaan resistor internal juga dapat menggantikan resistor eksternal.

2.4 Percobaan 4: Debouncing pada Input Tombol GPIO

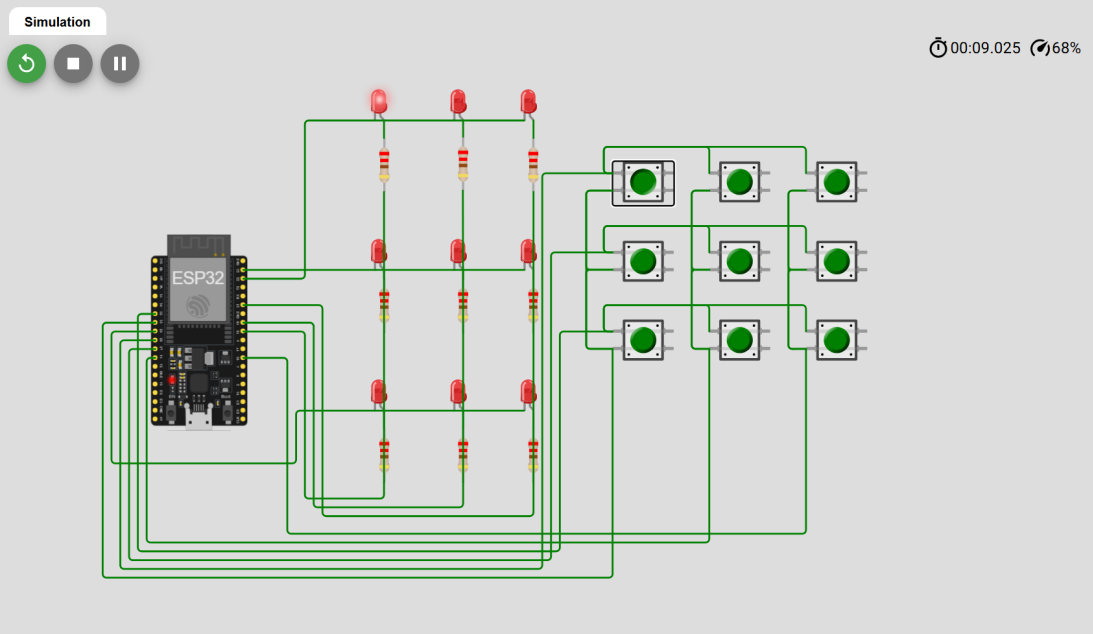
Pengujian menunjukkan bahwa debouncing membantu mengurangi pembacaan tombol yang berulang akibat bouncing. Perubahan nilai debounce delay juga memengaruhi respons tombol, terutama ketika nilai jeda terlalu kecil atau terlalu besar.

2.5 Percobaan 5: Pembacaan ADC dengan LDR

Nilai ADC berubah mengikuti intensitas cahaya yang diterima LDR. Perubahan tersebut juga memengaruhi nilai tegangan hasil konversi. Hal ini menunjukkan bahwa ESP32 dapat membaca perubahan sinyal analog melalui ADC.

3 Hasil Tugas Modul

3.1 Simulasi



Gambar 1: Simulasi Matrix Pushbutton 3x3 Mengendalikan Matrix LED 3x3

Listing 1: Source Code Matrix Pushbutton 3x3

```
#include <Arduino.h>

// LED matrix GPIO
const int ledCols[3] = {18, 19, 21};
const int ledRows[3] = {22, 23, 25};

// Pushbutton matrix GPIO
const int buttonRows[3] = {26, 27, 32};
const int buttonCols[3] = {33, 14, 16};

const unsigned long DEBOUNCE_MS = 40;

bool rawPressed[3][3] = {false};
bool stablePressed[3][3] = {false};
unsigned long lastChange[3][3] = {0};

void setup() {
    Serial.begin(115200);

    for (int col = 0; col < 3; col++) {
        pinMode(ledCols[col], OUTPUT);
        digitalWrite(ledCols[col], LOW);
    }

    for (int row = 0; row < 3; row++) {
        pinMode(ledRows[row], OUTPUT);
        digitalWrite(ledRows[row], HIGH);
```

```
    }

    for (int col = 0; col < 3; col++) {
        pinMode(buttonCols[col], INPUT_PULLUP);
    }

    for (int row = 0; row < 3; row++) {
        pinMode(buttonRows[row], INPUT);
    }

    Serial.println("3x3 Button and LED Matrix Ready!");
}

void scanButtons() {
    for (int row = 0; row < 3; row++) {
        for (int r = 0; r < 3; r++) {
            pinMode(buttonRows[r], INPUT);
        }

        pinMode(buttonRows[row], OUTPUT);
        digitalWrite(buttonRows[row], LOW);
        delayMicroseconds(50);

        for (int col = 0; col < 3; col++) {
            bool reading = (digitalRead(buttonCols[col]) == LOW);

            if (reading != rawPressed[row][col]) {
                rawPressed[row][col] = reading;
                lastChange[row][col] = millis();
            }

            if (millis() - lastChange[row][col] >= DEBOUNCE_MS) {
                if (stablePressed[row][col] != rawPressed[row][col]) {
                    stablePressed[row][col] = rawPressed[row][col];

                    Serial.print("Button (");
                    Serial.print(row + 1);
                    Serial.print(", ");
                    Serial.print(col + 1);
                    Serial.print("): ");

                    Serial.println(
                        stablePressed[row][col] ? "PRESSED" : "RELEASED"
                    );
                }
            }
        }

        pinMode(buttonRows[row], INPUT);
    }
}

void refreshLEDMatrix() {
    for (int row = 0; row < 3; row++) {
```

```
82          for (int r = 0; r < 3; r++) {
83              digitalWrite(ledRows[r], HIGH);
84          }
85
86          for (int col = 0; col < 3; col++) {
87              digitalWrite(ledCols[col], LOW);
88          }
89
90          digitalWrite(ledRows[row], LOW);
91
92          for (int col = 0; col < 3; col++) {
93              if (stablePressed[row][col]) {
94                  digitalWrite(ledCols[col], HIGH);
95              }
96          }
97
98          delayMicroseconds(1000);
99      }
100 }
101
102 void loop() {
103     scanButtons();
104     refreshLEDMatrix();
105 }
```

3.2 Analisis

1. **Mengapa debouncing tetap diperlukan pada rangkaian matrix pushbutton?**

   Debouncing tetap diperlukan karena tombol mekanis dapat mengalami bouncing, yaitu perubahan sinyal HIGH dan LOW secara berulang dalam waktu singkat saat tombol ditekan atau dilepaskan. Hal ini disebabkan oleh kontak logam di dalam tombol yang tidak langsung stabil. Jika tidak menggunakan debouncing, satu kali penekanan dapat terbaca sebagai beberapa kali penekanan sehingga LED dapat berubah status secara tidak sesuai.

2. **Bagaimana konfigurasi pull-up pada kolom tombol dan mengapa tidak boleh floating?**

   Rangkaian menggunakan konfigurasi internal pull-up (`INPUT_PULLUP`) pada setiap pin kolom tombol. Saat tombol tidak ditekan, pin akan berada pada kondisi HIGH, sedangkan saat tombol ditekan, pin akan terbaca LOW karena terhubung ke baris yang sedang diaktifkan. Kondisi floating harus dihindari karena tegangan pin menjadi tidak pasti sehingga pembacaan tombol dapat berubah secara tidak menentu.

3. **Apa perbedaan pembacaan digital input dan analog input menggunakan ADC?**

   Digital input hanya membaca dua kondisi logika, yaitu HIGH dan LOW, sehingga sesuai untuk membaca tombol. Sementara itu, analog input menggunakan ADC untuk mengubah tegangan analog menjadi nilai digital dengan beberapa tingkatan. Pada ESP32 dengan ADC 12-bit, hasil pembacaan memiliki rentang 0–4095. Oleh karena itu, ADC dapat merepresentasikan berbagai tingkat intensitas cahaya dari LDR, bukan hanya dua kondisi seperti digital input.

4 Kesimpulan

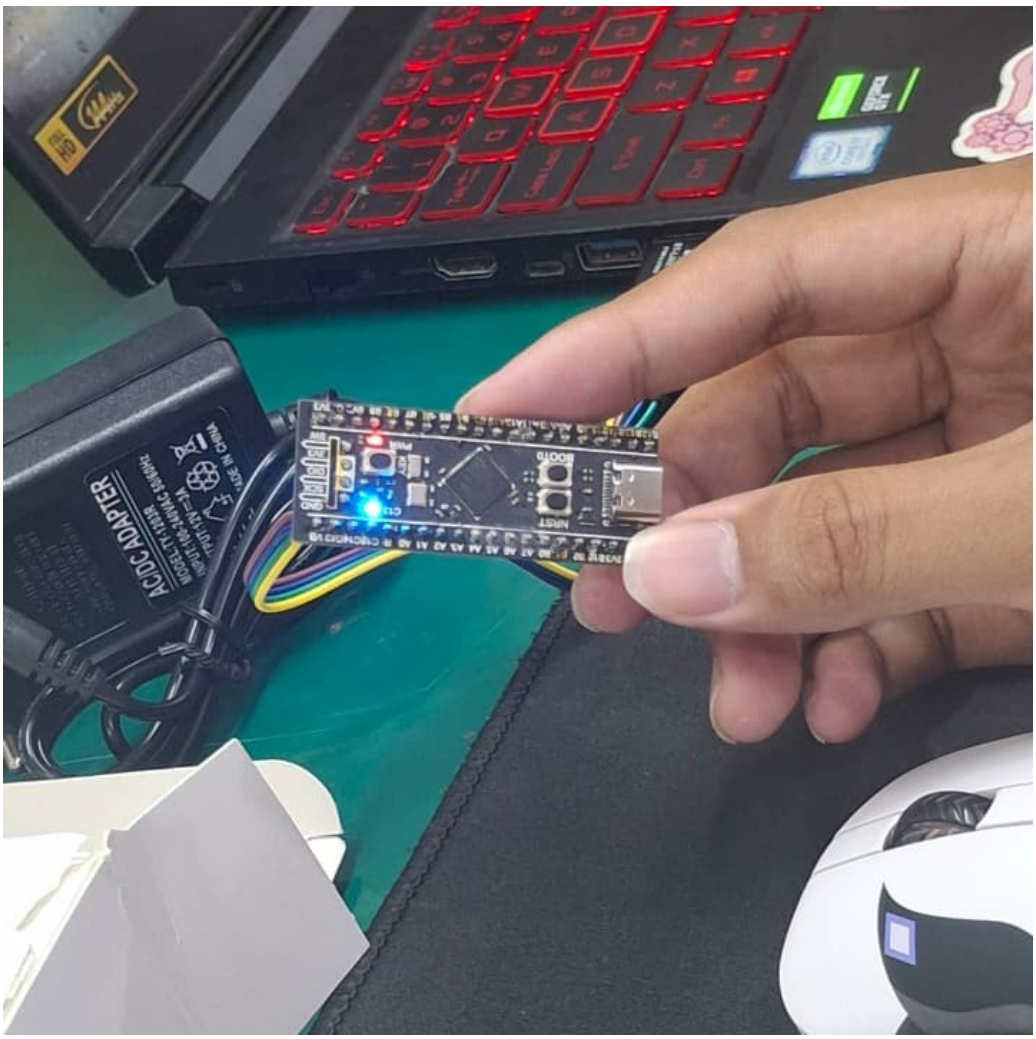
Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa praktikan telah mempelajari dasar penggunaan mikrokontroler STM32 dan ESP32 melalui pemrograman dan simulasi menggunakan PlatformIO. Praktikan juga memahami penggunaan GPIO sebagai input dan output, konfigurasi pull-up dan pull-down, serta penerapan debouncing untuk mengatasi perubahan sinyal akibat bouncing pada tombol.

Melalui percobaan ADC, praktikan mempelajari cara membaca sinyal analog dari sensor LDR dan mengubahnya menjadi nilai digital yang dapat diproses oleh mikrokontroler. Selain itu, pada tugas modul, praktikan berhasil membuat simulasi Matrix Pushbutton 3x3 yang mengendalikan Matrix LED 3x3. Setiap tombol dapat menyalakan LED pada posisi yang sesuai setelah pembacaan tombol melewati proses debouncing.

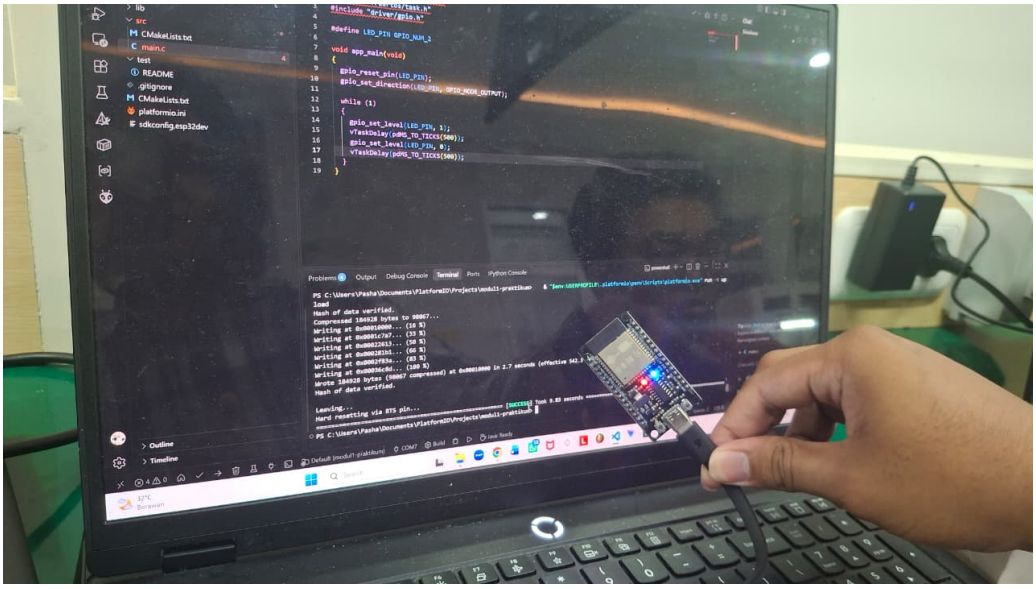
Secara keseluruhan, hasil praktikum sesuai dengan teori yang telah dipelajari. Praktikum ini memberikan pemahaman mengenai cara kerja mikrokontroler, pengolahan sinyal digital dan analog, serta penerapan konsep tersebut dalam sebuah sistem embedded sederhana.

5 Lampiran

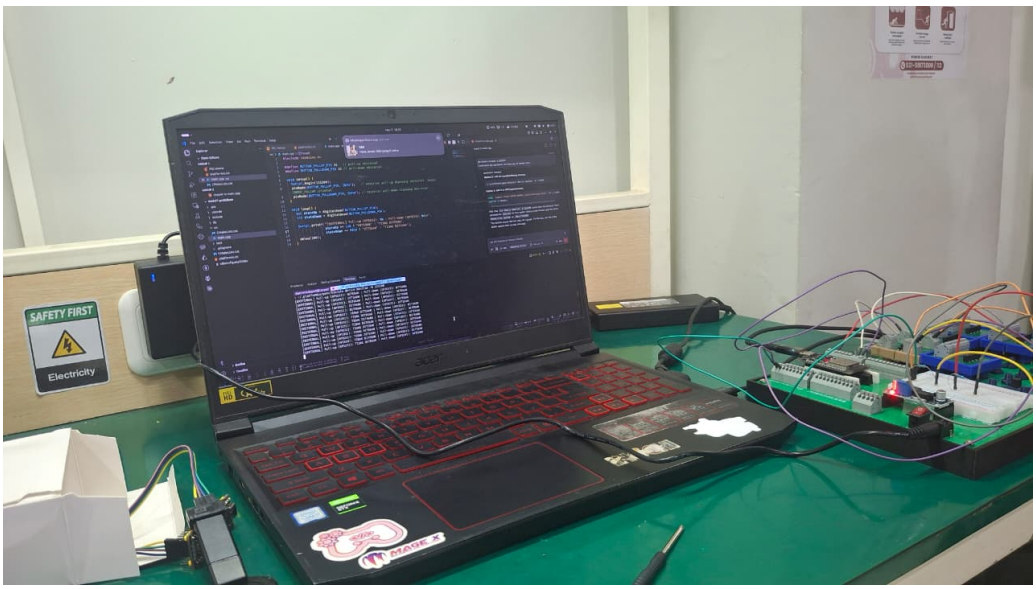
Lampiran ini berisi dokumentasi selama pelaksanaan praktikum, meliputi foto alat yang digunakan dan foto praktikan sebagai bukti pelaksanaan kegiatan praktikum.



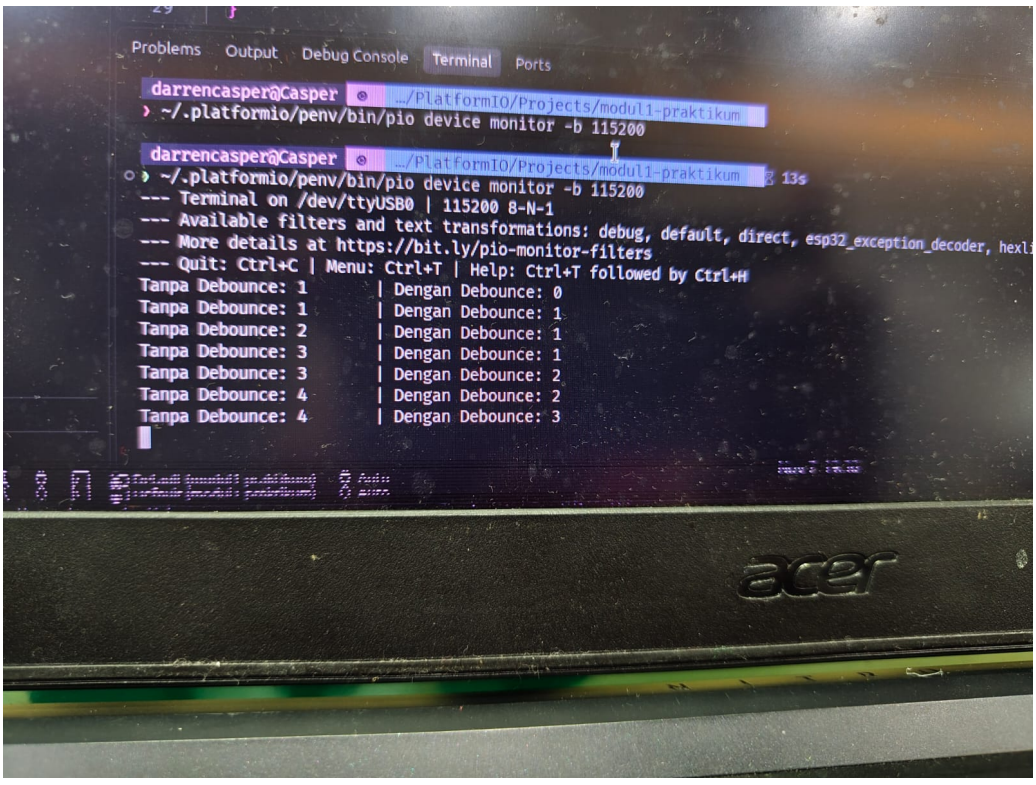
Gambar 2: Dokumentasi Praktikum 1



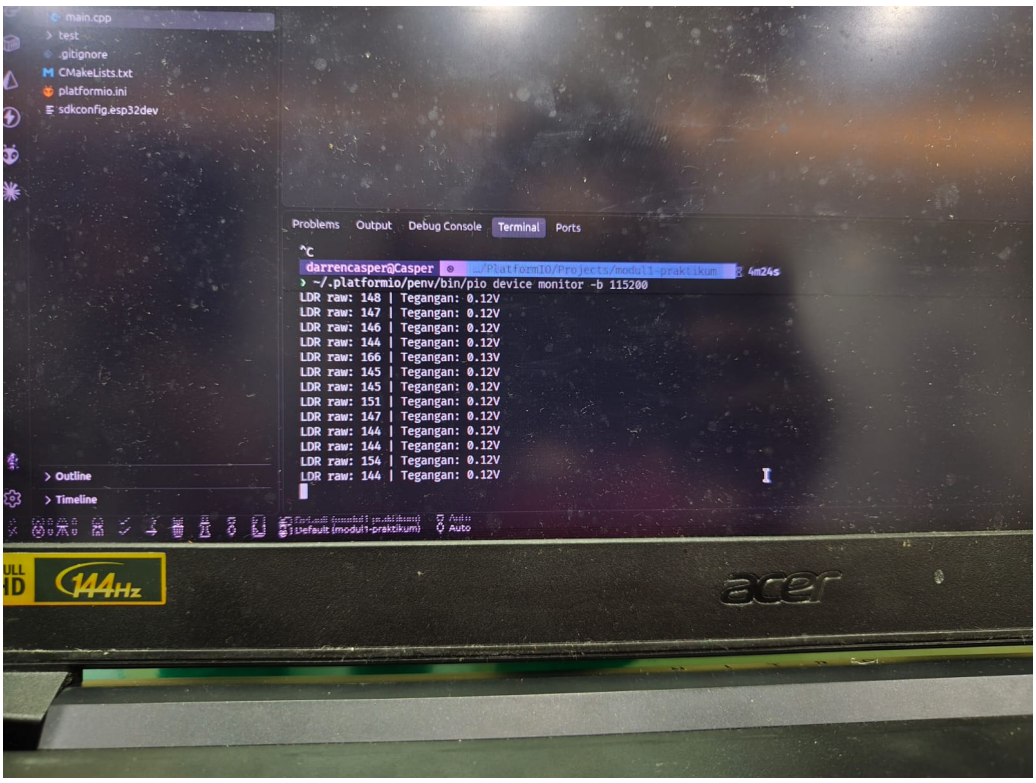
Gambar 3: Dokumentasi Praktikum 2



Gambar 4: Dokumentasi Praktikum 3



Gambar 5: Dokumentasi Praktikum 4



Gambar 6: Dokumentasi Praktikum 5